How the Internet Delivers a Web Page

A Case Study of www.google.com

Module 1 Assignment — PCCST501: Computer Networks

Name	Roll No.	Class
Yohan Abraham	61	S5 CSE A

Introduction

This report explains, step by step, the complete process that takes place when a user opens a web browser, types the URL "www.google.com", and presses Enter. It covers the underlying TCP/IP protocol stack, the role of DNS, how HTTP requests and responses work, how TCP guarantees reliable delivery, the client–server nature of the application, a comparison between HTTP and FTP, the role of cookies, and a visual flow of the entire communication.

1. The TCP/IP Protocol Stack

The Internet operates on a layered model known as the TCP/IP protocol stack. Each layer has a distinct responsibility, and data passed down the stack is progressively encapsulated with header information before being transmitted over the network.

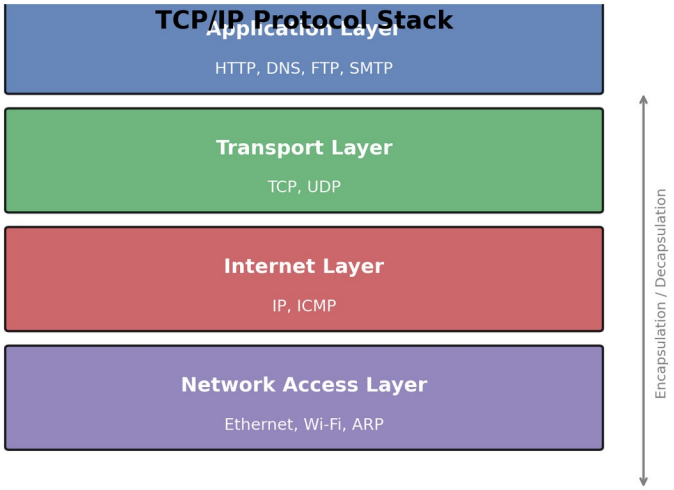


Figure 1: The TCP/IP Protocol Stack

Role of each layer

- Application Layer: Provides network services directly to end-user applications. When accessing Google, this layer generates the HTTP request and uses DNS to resolve the domain name.
- Transport Layer: Uses TCP to break data into segments, number them, and ensure reliable, ordered, and error-checked delivery between the browser and the web server.
- Internet Layer: Uses IP to logically address each packet with source and destination IP addresses and route it across interconnected networks toward the destination.
- Network Access Layer: Handles the physical transmission of frames over the actual medium (Wi-Fi, Ethernet), including MAC addressing and error detection at the link level.

2. Role of DNS

A user does not naturally remember a website's numeric IP address, so the Domain Name System (DNS) acts as the Internet's phonebook. When the browser needs to reach “www.google.com”, it sends a DNS query that is resolved through a hierarchy: a DNS resolver (usually run by the ISP) first checks its cache; if not found, it queries a root server, then a Top-Level Domain (.com) server, and finally the authoritative name server for google.com, which returns the correct IP address (for example, 142.250.195.100). This IP address is what the browser actually uses to open a connection to Google's web server.

3. HTTP: Requesting and Receiving the Webpage

Once the IP address is known, the browser uses the Hypertext Transfer Protocol (HTTP), or HTTPS for a secured version, to communicate with the web server. The browser sends an HTTP GET request containing the requested resource path, headers (such as browser type and accepted content formats), and any cookies. The web server processes the request and returns an HTTP response consisting of a status code (e.g., 200 OK), response headers, and the actual content (HTML, CSS, JavaScript, and images) that the browser then renders as the visible webpage.

4. How TCP Provides Reliable Communication

HTTP relies on the Transmission Control Protocol (TCP) beneath it to guarantee reliable delivery of the webpage data. Before any data is exchanged, the client and server perform a three-way handshake (SYN, SYN-ACK, ACK) to establish a connection. TCP then divides the data into numbered segments, uses acknowledgements to confirm receipt, retransmits any lost or corrupted segments, and reassembles the segments in the correct order at the destination. It also uses flow control and congestion control to avoid overwhelming the network or the receiver, ensuring that the complete and correct webpage content reaches the browser.

5. Client–Server or Peer-to-Peer Architecture?

Google follows a Client–Server architecture. The browser (client) always initiates the request, and Google's web servers (server) respond by providing the requested resources; the client never acts as a resource provider to other clients. This is justified because: (a) the service is centrally hosted on Google's own infrastructure and data centers, (b) all users connect to the same set of servers rather than sharing content directly with one another, and (c) the server holds sole authority over the content, search index, and processing logic, which is a defining characteristic of the client–server model, unlike Peer-to-Peer (P2P) systems where each node can act as both client and server.

6. HTTP vs. FTP — Comparison

Although both HTTP and FTP are application-layer protocols built on TCP, they serve different purposes and behave differently, as summarized below:

Aspect	HTTP	FTP
Purpose	Transfers web pages (HTML, CSS, JS, images) for browsing	Transfers files between a client and server
Connections used	Uses a single connection for request and response	Uses two separate connections: control (port 21) and data (port 20)
Statefulness	Stateless by default; each request is independent (cookies add state)	Stateful; maintains a session throughout the transfer
Typical port	80 (HTTP) / 443 (HTTPS)	21 (control) and 20 (data)
Use case example	Loading a webpage such as google.com	Uploading files to a website's server via an FTP client

7. Role of Cookies

Cookies are small pieces of data that a web server sends to the browser, which the browser stores and sends back with subsequent requests to the same site. Since HTTP is inherently stateless, cookies allow the server to “remember”

information about a user across multiple requests, improving the user experience through personalization and session continuity.

Real-world example: When a user logs into their Google account, a session cookie is stored in the browser. This cookie allows the user to navigate between Gmail, YouTube, and Google Search while remaining logged in, without needing to re-enter credentials on every single page or request.

8. Flow Diagram: Browser-to-Web-Server Communication

The diagram below summarizes the end-to-end communication flow, starting from DNS resolution and ending with the browser rendering the returned webpage.

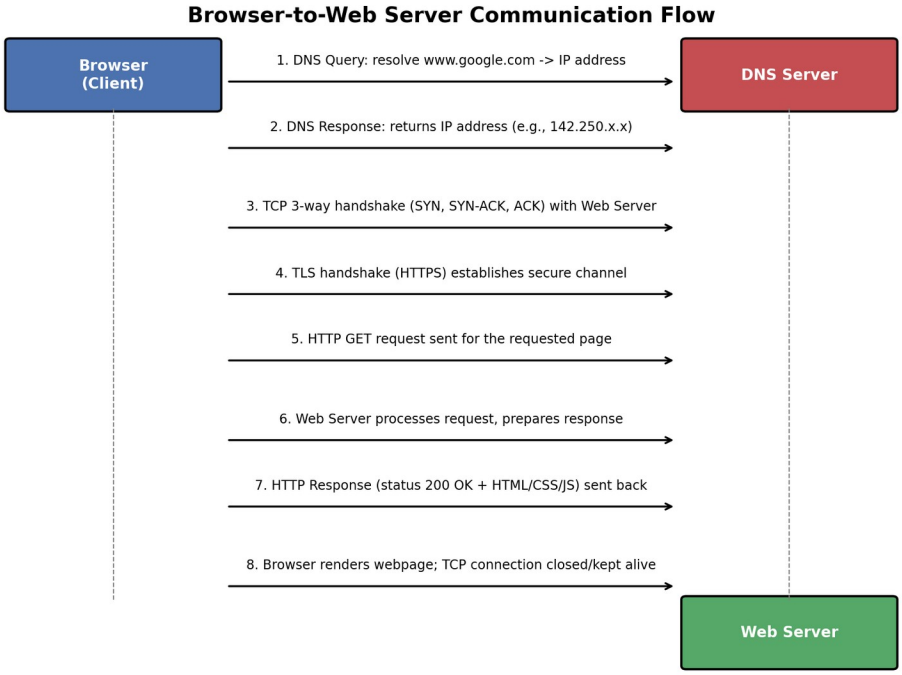


Figure 2: Complete Communication Flow from Browser to Web Server and Back

9. Conclusion

Application-layer protocols such as HTTP, DNS, and FTP form the backbone of everyday Internet usage, quietly enabling actions as simple as opening a search engine or streaming a video. Every visit to a website like Google involves a coordinated sequence of events: DNS resolves a human-readable name into a machine-usable IP address, TCP establishes a reliable channel through a handshake and guarantees ordered, error-free delivery, and HTTP structures the actual exchange of the requested content between browser and server. Supporting mechanisms like cookies further enhance usability by preserving state across a stateless protocol. Understanding this layered process highlights why application-layer protocols are indispensable: they translate complex, low-level networking operations into the simple, near-instantaneous experience users rely on daily, making the modern Internet both functional and user-friendly.